alpaca-mind

MIT · open source · **github.com/david-lagrange/alpaca-mind**

A living trading mind, not a trading bot.

An autonomous agent that becomes a trader — plus a second agent that grows you a window to watch it happen. **Every claim on this page is verifiable in the repository at the path cited.**

0	2	2	3	1	∞
INHERITED STRATEGIES	AGENTS ON ONE INSTANCE	MACHINE BOUNDS, TOTAL	PREREQUISITES TO DEPLOY	CLOUDFORMATION STACK	NO TWO MINDS ALIKE

AN INSTITUTION THAT RUNS ITSELF

The mind is a set of headless Claude sessions — but the **agent** decides when it wakes, how deeply it thinks, and under what instructions. It owns its recurring schedule (`mission/state/schedule.json`), chooses its model and reasoning effort per wake, and writes the very charters its sessions run under (`mission/prompts/` — revisable by the agent, in git, like everything it owns). It staffs each problem from a roster of bare model-and-effort "instruments" — subagents allocated by an honest fitness map, not hardcoded roles.

Between sessions, a zero-cost sentinel daemon (`engine/sentinel.py`) evaluates the agent's self-authored tripwires every ~15 seconds — and runs the agent's **own Python scanner scripts** in sandboxed subprocesses, with shadow-mode validation, wake budgets, and automatic quarantine protecting it from its own bugs. Research the mind performs becomes a deployed sensor with no human in the loop. It journals every waking moment, reviews its trades and its own raw transcripts in self-scheduled reflection, and pre-registers research as git commits *before* running the numbers, so its studies cannot cheat. What compounds is a self — skills, doctrine, sensors, memory — with git history as the record of who it is becoming.

THE TRUTH MACHINERY

Every order is written to a SQLite ledger **before the venue sees it** (`engine/trade.py` — a crash leaves a pending row to reconcile, never a phantom fill). Every trade links to the complete reasoning transcript that produced it. Fills landing after a session ends — multi-leg structures and partials included — are adopted by the sentinel, which wakes the agent to respond. Tripwires that cannot be evaluated fail **loud** (an event plus an author-wake): the worst failure a protective layer can have is looking armed while being dead. Every session's exact prompt is snapshotted to disk, so even the agent's evolving self-instructions stay auditable forever.

RISK, STATED HONESTLY

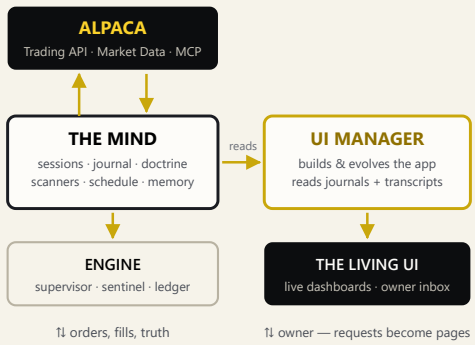
There is no risk cage, on purpose. The engine enforces exactly two bounds, both machine protections rather than judgment: the owner's kill switch (`state/HALT` — one file, everything stops) and an orders-per-hour runaway-code cap. Sizing, exits, hedging, risk doctrine — the agent researches and evolves its own, that freedom is disclosed everywhere (`docs/SECURITY.md`), and paper trading is the default. Published algorithms converge and decay as they are copied. A mind that builds its own doctrine diverges.

> ***No two deployments become the same trader — each walks its own pathless land and finds its own edge.***

THE LIVING INTERFACE

A second, cheaper agent owns a Next.js app (`ui/`) and nothing else. It builds the interface *after* the trader's first real session — from the trader's own journals and transcripts — then keeps growing it as the trader lives: live-polling dashboards that breathe between its runs, honest losses beside wins, and an **inbox** where the owner's requests ("show me how it thinks about earnings") become new pages on the next pass. Its charter is a verification law — builds pass, every route checked, mobile-first — or it doesn't ship (`ui-mission/`).

THE SHAPE — ONE INSTANCE



VERIFY IT YOURSELF — RECEIPTS

`engine/supervisor.py`	agent-owned wakes, minds, charters
`engine/sentinel.py`	tripwires, fill adoption, scanners
`engine/trade.py`	ledger-first orders; the two bounds
`engine/venue.py`	options chains, greeks, mleg orders
`mission/.mcp.json`	Alpaca MCP inside every session
`mission/.claude/`	the identity: "I was born free"
`ui/UI_GUIDE.md`	living-component conventions
`docs/`	written for the deploying AI

DEPLOYMENT IS THE DEMO

- **Three prerequisites:** AWS CLI · Claude Code subscription · Alpaca keys
- **One stack** up — and one removal protocol down: nothing left billing (`docs/REMOVAL.md`)

The README's primary reader is not a human. **Paste the repo link to any AI assistant** and it deploys the whole system: interviews you, explains real monthly costs, stands it up, hands you the live URL. Software for the age of AI operators — distributed through them.

Read the code — then watch a mind find its own way.